Correctness Assessment of Code Generated by Large Language Models Using Internal Representations

Tuan-Dung Bui, Thanh Trong Vu, Thu-Trang Nguyen*, Son Nguyen and Hieu Dinh Vo

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:
LLM-generated code, code LLMs, code quality assessment, internal representations, white-box, open-box approach

ABSTRACT

Ensuring the correctness of code generated by Large Language Models (LLMs) presents a significant challenge in AI-driven software development. Existing methods predominantly rely on black-box (closed-box) approaches that evaluate correctness post-generation failing to utilize the rich insights embedded in the LLMs' internal states during code generation. This limitation leads to delayed error detection, increased debugging costs, and reduced reliability in deployed AI-assisted coding workflows. In this paper, we introduce OPENIA, a novel white-box (open-box) framework that leverages these internal representations to assess the correctness of LLM-generated code. By systematically analyzing the intermediate states of representative open-source code LLMs, including DeepSeek-Coder, CODE LLAMA, and MAGICODER, across diverse code generation benchmarks, we found that these internal representations encode latent information, which strongly correlates with the correctness of the generated code. Building on these insights, OPENIA uses a white-box/open-box approach to make informed predictions about code correctness, offering significant advantages in adaptability and robustness over traditional blackbox methods and zero-shot approaches. Our results show that OPENIA consistently outperforms baseline models, achieving higher accuracy, precision, recall, and F1-Scores with up to a 2X improvement in standalone code generation and a 3X enhancement in repository-specific scenarios. By unlocking the potential of in-process signals, OPENIA paves the way for more proactive and efficient quality assurance mechanisms in LLM-assisted code generation.

1. Introduction

The emergence of **L**arge **L**anguage **M**odels for Code (Code LLMs) has transformed the landscape of automated code generation, offering a promising solution to improve developer productivity and address the growing demand for software [1, 2, 3, 4]. These models, trained on vast repositories of code, are capable of automating repetitive tasks, suggesting optimizations, and even generating complex functionalities. As LLM-generated code becomes increasingly integrated into real-world systems, ensuring the quality and reliability of this code has become critical [5, 6, 7, 8]. Low-quality code can lead to significant functional failures, severe security vulnerabilities, and increased maintenance costs, emphasizing the need for rigorous evaluation and quality assurance mechanisms [9].

However, recent studies show a concerning trend that LLM-generated code frequently contains more bugs and security vulnerabilities than human-written code [7, 8, 10, 11, 12, 13, 14, 5]. This problem has led developers to find LLM-generated code often unusable, resulting in considerable time spent debugging or discarding it entirely [1, 15]. To detect and mitigate these errors, current approaches rely on post-hoc methods, such as static or dynamic code analysis and testing, which can be resource-intensive and may miss context-specific issues.

*Corresponding author

21020006@vnu.edu.vn (T. Bui); thanhvu@vnu.edu.vn (T.T. Vu); trang.nguyen@vnu.edu.vn (T. Nguyen); sonnguyen@vnu.edu.vn (S. Nguyen); hieuvd@vnu.edu.vn (H.D. Vo)

ORCID(s): 0009-0007-7318-6896 (T. Bui); 0000-0002-3596-2352 (T. Nguyen); 0000-0002-8970-9870 (S. Nguyen); 0000-0002-9407-1971 (H.D. Vo)

Unlike post-hoc approaches analyzing the final code only after code generation is *complete*, an *in-process* strategy takes advantage of the internal states capturing the "thought process" of code LLMs *during* code generation, enabling early detection of potential issues. This is particularly promising for code LLMs, which offer a unique "*open-box*" advantage to access code LLMs' internal states, especially with open-source code LLMs [16, 17, 18]. Unlike human coding, which remains a "*closed-box*" process where only the final code is visible, and the developer's intermediate reasoning and thought process are inaccessible, code LLMs generate a wealth of internal representations and byproducts that can be examined in real-time. By leveraging these signals, we can move beyond passive validation and instead, proactively assess correctness as code is generated. This opens an exciting opportunity: *Can we leverage this visibility to proactively detect during the code generation process, preventing bugs before they reach developers?*

While extensive research has been conducted on the performance and capabilities of code LLMs [7, 8, 19, 20, 21, 22], there is a notable gap in understanding how their internal representations, specifically the hidden states during code generation, encode information related to reliability. These internal states capture the model's intermediate understanding of the input and output, but their relationship with code quality remains largely unexplored. If these representations do indeed encode reliability signals, they could offer a novel pathway to assess and enhance code quality directly during generation rather than relying solely on external validation.

We hypothesize that *the internal states of LLMs encapsulate meaningful signals that reflect the correctness of the code they generate*. These signals, embedded in the

representations formed during the generation process, may correlate with key quality attributes. Understanding and leveraging these signals could enable the proactive identification of faults, thereby improving the overall quality assurance pipeline.

In this work, we tackle the challenges and gaps in assessing the reliability of code generated by LLMs by presenting two key contributions. ***First***, we introduce OPENIA, a novel framework that leverages the internal representations of code LLMs during code generation to assess the correctness of the generated code. Unlike traditional quality assurance techniques that rely heavily on post-hoc validation, OPENIA identifies and utilizes the signals embedded within the LLM's internal states. By doing so, it could enable earlier detection of potential bugs, reducing both the computational cost and the time required for quality assurance. ***Second***, we conduct a systematic analysis of the internal signals of code LLMs during code generation, focusing on three representative open-source models, including DeepSeek-Coder, CODE LLAMA, and MAGICODER. These models are evaluated across diverse benchmarks, encompassing standalone code generation and repository-level generation tasks. Our analysis reveals specific patterns and features in the internal representations that align with the reliability attributes of the generated code.

Specifically, our experimental results demonstrate that OPENIA consistently outperforms traditional classification-based and zero-shot approaches across a range of benchmarks. For independent/standalone code generation tasks, OPENIA achieves up to a 2X improvement in accuracy compared to zero-shot baselines and surpasses classification-based methods, such as those utilizing CodeBERT and CodeT5+, by up to 20% in accuracy. In repository-level code generation, OPENIA shows a remarkable, up to 3X, improvement in F1-Score, highlighting its robustness in handling complex and context-dependent coding scenarios. These findings provide valuable insights into how LLMs encode and process information related to code correctness, offering a deeper understanding of their inner workings.

By combining these contributions, this work lays the foundation for more efficient and proactive approaches to ensuring the quality of LLM-generated code. It highlights the latent capabilities of code LLMs to encode reliability-related information and demonstrates the potential of leveraging these signals to enhance quality assurance in automated software development.

2. Internal Representations in LLMs for Code Generation

Code LLMs rely on a sequence-to-sequence mapping mechanism, where input sequences (e.g., natural language descriptions or partially written code) are mapped to output sequences (e.g., complete or corrected code) [23, 19, 20, 24]. Central to this process is the internal representations, which encode the intermediate understanding of the model as it processes inputs and generates outputs. These representations provide a window into the code LLM's decision-making process, offering insights into how the model predicts the next code tokens.

2.1. The Code Generation Process

For a code LLM $\mathscr{M}$, given an input sequence $x = \{x_1, x_2, ..., x_n\}$, where x_i represents the i-th token in the input, $\mathscr{M}$ generates a code output sequence $c = \{c_1, c_2, ..., c_m\}$ token by token. At each generation step s, the model predicts the next code token c_s by leveraging both the input sequence and the tokens generated so far, denoted $c_{<s>} = \{c_1, c_2, ..., c_{s-1}\}$. The prediction process at step s can be represented as:

$$P(c_s|x, c_{<s}) = \text{softmax}(W_o \cdot h_{l,s} + b_o)$$

where $h_{l,s}$ is the final hidden state of the LLM at layer l for token c_s, W_o and b_o are the output layer weights and bias.

2.2. Internal Representations

Internal representations, or *intermediate activations*, are vectors $h_{l,s}$ that capture the latent information processed by the model at a specific layer l for a given code token c_s. These activations are computed as:

$$h_{l,s} = f_l(h_{l-1,s}, \text{context}_s)$$

where $h_{l-1,s}$ is the activation from the preceding layer $(l-1)$. Meanwhile, context_s represents contextual information (e.g., attention-weighted representations of other tokens), f_l is the layer-specific transformation function (e.g., feedforward, attention, or normalization operations) applied at layer l. Note that the initial hidden state for a token c_s is $h_{0,s}$ is the embedding vector e_s of c_s.

Each $h_{l,s}$ is a vector $\mathbb{R}^d$ where d is the dimensionality of the LLM's hidden states (e.g., 4,096 in the case of CODE LLAMA). The sequence of activations across all layers and code tokens forms a hierarchical representation of the input and generated sequences that could encode syntactic, semantic, and contextual information at varying levels of abstraction. Internal representations encapsulate the underlying structure and logic the model uses to generate code. By analyzing these representations, we gain insights into the model's "thought process" and "intuition" during code generation. These representations offer valuable cues about the correctness and reliability of the generated code c.

3. A Framework Leveraging Internal Representations for LLM Generated Code Quality Assessment

3.1. Task Formulation

Given a code LLM $\mathscr{M}$, an input prompt x, and the corresponding LLM-generated code c, the objective is to predict whether c meets the correctness criteria as defined by the requirements provided in x. Specifically, the correctness of c is evaluated by verifying whether c's behaviors align with the intended behaviors specified in x.

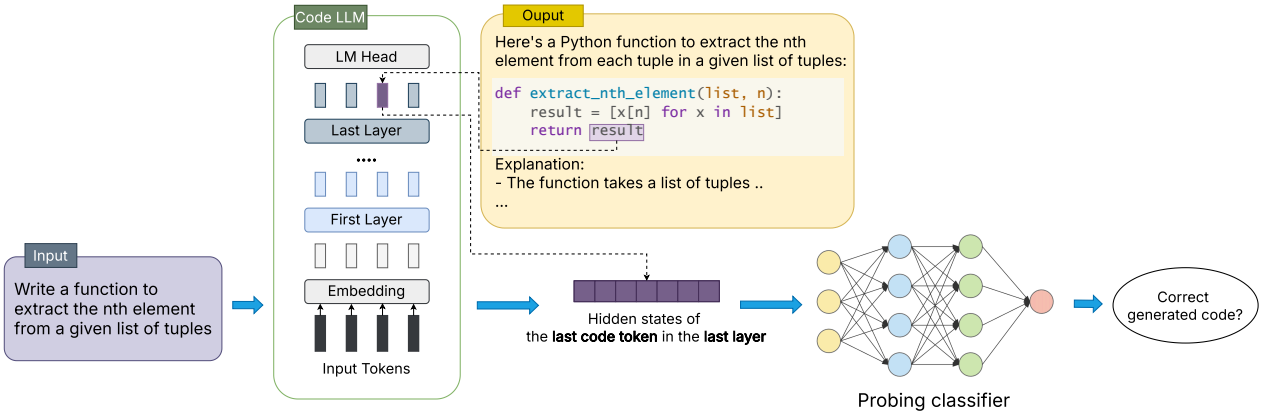


Figure 1: OPENIA's overview

Let $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ represent a set of validation tests designed to assess whether c adheres to the requirements in x. Each test $t \in \mathcal{T}$ returns an outcome, denoted as $t(c)$, which indicates whether c *passed* or *failed* the test. Code c is considered correct if and only if it passes all the tests.

$$\text{Correctness}(c) = \begin{cases} 1 & \text{if } \forall t \in \mathcal{T},\ t(c) = \textit{passed}, \\ 0 & \text{otherwise.} \end{cases}$$

The goal is to develop a predictive model $\mathcal{Q}$ that evaluates the correctness of generated code c based on the requirements specified by the input prompt x and the context provided by the code LLM $\mathcal{M}$. Formally:

$$\mathcal{Q}(\mathcal{M}, x, c) = y$$

where $y \in \{0, 1\}$ is the predicted correctness label for c, with $y = 1$ indicating that c meets the correctness criteria and $y = 0$ otherwise.

3.2. Framework Overview

Fig. 1 illustrates the overview of our framework, which leverages the internal representations of code LLMs during code generation to assess the correctness of its generated code. Specifically, given a code LLM $\mathcal{M}$ and its output code sequence $c = \{c_1, c_2, \ldots, c_m\}$, OPENIA extracts the all internal states produced during the decoding process of $\mathcal{M}$:

$$H = \left\{ h_{l,s} | l \in \{1, ..., L\}, s \in \{1, ..., m\} \right\}$$

where $h_{l,s}$ represents the hidden state at layer l corresponding to token c_s. These internal representations are then fed to a probing classifier [25], which determines whether the generated code c is correct.

Fig. 2 and Fig. 3 show the examples of prompts and corresponding responses generated by DEEPSEEK CODER-6.7B for the independent-unit and repository-level code generation tasks, respectively. Our goal is to utilize the model's internal representations produced during the generation process to predict the correctness of the generated code (Fig. 2b and Fig. 3b). These internal representations refer to the hidden states associated with each generated token across different layers of the model. For instance, during generation, each token (e.g., keywords like `for`, `if` or identifiers like `balance` in Fig. 2b) is represented by hidden states across all 32 transformer layers of DEEPSEEK CODER-6.7B. Suppose OPENIA chooses the *final code token*, i.e., `False` in Fig. 2b, as the focus of analysis. From this token, OPENIA extracts the corresponding hidden states at a particular layer, e.g., the last (32nd) layer, which captures high-level semantic features that reflect the model's overall reasoning about the function. These representations are then passed to a probing classifier to predict the likelihood that the generated function is correct. To obtain informative signals for code correctness prediction, OPENIA aims to select appropriate tokens and

```
from typing import List

def below_zero(operations: List[int]) -> bool:
    """ You're given a list of deposit and withdrawal operations on a bank account that starts with
    zero balance. Your task is to detect if at any point the balance of account falls below zero,
    and at that point function should return True. Otherwise it should return False.
    >>> below_zero([1, 2, 3])
    False
    >>> below_zero([1, 2, -4, 5])
    True
    """
```

(a) Prompt provided to the model

```
Here is a Python solution using a simple loop to sum up the operations
and check if the balance goes below zero at any point:

```python
from typing import List

def below_zero(operations: List[int]) -> bool:
    balance = 0
    for operation in operations:
        balance += operation
        if balance < 0:
            return True
    return False
```

This function works by iterating over each operation in the list. It
adds the operation to the current balance. If at any point the balance
is less than zero, the function immediately returns True. If the
function finishes iterating over all operations without finding a
negative balance, it returns False.
```

(b) Response generated by DEEPSEEK CODER-6.7B for the prompt in Fig. 2a.

Figure 2: An example of independent-unit code generation for a task in the *HumanEval* benchmark

```
Please complete the is_json_serializable function in the middle of a file.
The contexts above the function are:

```Python
import pathlib
import re
from datetime import datetime
from decimal import Decimal
...
def is_bool(val):
    return isinstance(val, bool)
def is_collection(val):
    return isinstance(val, (dict, list, set, tuple))
...
```Python

def is_json_serializable(val):

Check if the input value is JSON serializable. It checks if the input value is of the JSON
serializable types. Input-Output Arguments    :param val: Any. The input value to be checked for
JSON serializability. :return: Bool. True if the input value is JSON serializable, False otherwise.

Completed code:
```

(a) Prompt provided to the model

```Python
def is_json_serializable(val):
    """
    Check if the input value is JSON serializable. It checks if the input value is of the JSON
serializable types.
    Input-Output Arguments
    :param val: Any. The input value to be checked for JSON serializability.
    :return: Bool. True if the input value is JSON serializable, False otherwise.
    """
    if is_bool(val) or is_integer(val) or is_float(val) or is_string(val) or is_none(val):
        return True
    elif is_list(val):
        return all(is_json_serializable(item) for item in val)
    elif is_dict(val):
        return all(is_string(key) and is_json_serializable(val) for key, val in val.items())
    elif is_tuple(val) or is_set(val):
        return all(is_json_serializable(item) for item in val)
    elif is_datetime(val) or is_decimal(val) or is_uuid(val):
        return True
    else:
        return False
```
```

(b) Response generated by DEEPSEEK CODER-6.7B for the prompt in Fig. 3a.

**Figure 3:** An example of repo-level code generation for a task in *DevEval* benchmark

extract their corresponding internal representations from a suitable layer within the model.

**Layer Selection for Internal Representations**. Modern (code) LLMs often contain multiple transformer layers, each contributing uniquely to the model's ability to understand and process various tasks. For example, DeepSeek-Coder-6.7B [16] and CODE LLAMA-7B [17] comprise 32 layers, while the 34B version of CODE LLAMA consists of 60 layers. Each layer captures different levels of abstraction and semantic representation; shallow layers often encode lower-level syntactic and lexical features, while deeper layers focus more on abstract and semantic representations. Jin *et al.* [26] have demonstrated the alignment of shallow and deep layers with simpler and more complex tasks, respectively. Meanwhile, Song *et al.* [27] highlights the varying importance of layers within an LLM. These studies emphasize that not all layers contribute equally to a given task. Thus, *selecting the appropriate layer(s)* among $L$ layers of $\mathscr{M}$ for extracting internal representations is crucial for achieving optimal performance in downstream tasks. An inappropriate choice of layer may fail to capture the necessary information, leading to suboptimal results. In this work, we conduct several experiments to systematically evaluate the impact of layer selection on the task of assessing the correctness of LLM-generated code (Sec. 5.2.1).

**Token Selection for Internal Representations**. In addition to layer selection, *the choice of specific tokens* from which internal representations are extracted is another critical factor. Valuable signals for correctness assessment can reside in several specific tokens within the generated output or its preceding input. Several works [28, 29, 30] have highlighted the effectiveness of probing the hidden states of the final token in the generated answer to assess the answer's correctness. This token typically encapsulates the accumulated context and semantics of the entire generated sequence. Therefore, the hidden states of this token can provide meaningful insights for determining whether the answer is correct or not. Meanwhile, some other approaches [31, 32] focus on the final token of the prompt, just before the response is generated. The reasoning behind this choice is that this token often reflects the model's transition from input comprehension to output generation, making its hidden states indicative of how well the model has understood the task requirement and its readiness to generate the output. In this work, we also systematically investigate the impact of different token selection strategies on assessing the correctness of the generated code. Our goal is to understand how token-level choices affect the effectiveness of OPENIA and to explore the relationship between token-specific hidden states and the correctness of the generated code (Sec. 5.2.2).

*In summary*, our approach, OPENIA, investigates how internal representations of LLMs can be effectively utilized to assess the correctness of generated code. OPENIA employs a representation probing classifier, which leverages these internal representations to predict whether the generated code is correct. By systematically analyzing layer selection and token selection, we aim to identify the most informative internal representations that contribute to robust and accurate correctness assessment. Through empirical evaluations, we demonstrate how different layers and token positions impact the effectiveness of correctness assessment, providing insights into optimizing internal representation selection for improved downstream task performance.

## 4. Evaluation Design

In this research, we seek to answer the following research questions:

**RQ1. Performance Analysis:** To what extent can OPENIA leverage the internal representations of an LLM to assess the correctness of its generated code? How accurately does OPENIA compare to the baseline approaches?

**RQ2. Layer and Token Selection Analysis:** How do the representations extracted from different token positions and layer depths impact OPENIA's performance?

**RQ3. Sensitivity Analysis:** How do different factors such as programming languages and task difficulty levels impact OPENIA's performance?

**RQ4. Time Complexity:** What is OPENIA's running time?

### 4.1. Dataset Construction

OPENIA constructs the dataset using three popular benchmarks, *HumanEval* [33], *MBPP* [34], and *DevEval* [35]. In particular, HumanEval and MBPP contain 164 and 500 standalone programming problems, respectively, while DevEval consists of 1,825 tasks of repository-level code generation. For each task in these benchmarks, we employed each studied model to generate 10 candidate solutions. The correctness of these generated solutions is then evaluated
```

Table 1
The numbers of correct (*#Cor.*) and incorrect (*#Inc.*) solutions of benchmarks

Code LLM		*HumanEval*	*MBPP*	*DevEval*
DeepSeek Coder-*1.3B*	*#Cor.*	872	2,065	2,092
	#Inc.	768	2,701	15,237
DeepSeek Coder-*6.7B*	*#Cor.*	1,294	3,033	4,321
	#Inc.	346	1,749	13,009
Code Llama-*7B*	*#Cor.*	644	1,942	3,467
	#Inc.	996	2,686	13,863
Code Llama-*13B*	*#Cor.*	666	2,065	2,849
	#Inc.	974	2,855	14,481
Magicoder-*7B*	*#Cor.*	1,198	3,135	4,113
	#Inc.	440	1,752	13,217

using the test cases provided by the respective benchmarks. Table 1 summarizes the number of correct and incorrect solutions generated by each model for the given benchmarks. The complete dataset, including all generated code, the corresponding internal representations extracted from the studied code LLMs, and their correctness labels could be found on our website [36].

4.2. Studied Code LLMs

To ensure a fair, reproducible, and practical evaluation, we carefully selected LLMs specialized for code based on the following criteria. First, the models must be open-source to enable access to their internal representations during inference. Second, the models should be trained or fine-tuned on large-scale code corpora, as code-specialized LLMs are better suited for programming tasks and widely used in practice. Third, we limited the model size to 13B parameters to accommodate hardware resource constraints. These criteria were designed to balance model performance, accessibility, and practical usability.

Following these criteria, we selected five representative models: DEEPSEEK CODER-1.3B [16], DEEPSEEK CODER-6.7B [16], CODE LLAMA-7B [17], CODE LLAMA-13B [17], and MAGICODER-7B [18]. These models are all open-source, widely adopted in both the research and industry, and have demonstrated strong performance in various code generation tasks. Each model employs advanced architectures and training methodologies designed to capture the complexities of programming languages, including syntax, semantics, and logical consistency. For consistency across experiments, we use their instruction-tuned variants with officially released pre-trained weights from HuggingFace and do not perform any additional fine-tuning. This setup allows us to assess the models' out-of-the-box generalization capabilities and supports a controlled comparison of how internal representations relate to code correctness.

4.3. Experimental Procedure

RQ1. Performance analysis:

Baselines: We evaluated the performance of OPENIA by comparing it with three baseline approaches:

- *Inference-time representation-based methods* [37, 38, 39, 40]: These approaches leverage the internal representations of LLMs during inference to access the correctness of generated outputs. To the best of our knowledge, no prior work has specifically explored internal representations for evaluating the correctness of code generated by LLMs. For comparative evaluation, we adopt LLM-CHECK [40], a method originally proposed for assessing the quality of LLM-generated outputs in natural language question answering tasks. Specifically, LLM-CHECK utilizes the diversified scoring methods, including internal attention kernel maps, hidden activations and output prediction probabilities, from different model components to maximize the capture of hallucinations.

- *Post-hoc classification-based methods* [41, 42, 43]: These approaches focus on capturing the semantics of generated code to predict its correctness. Specifically, pre-trained models such as CodeBERT [44] and CodeT5 [45] or CodeT5+ [46] are employed to encode the semantics of the LLM-generated code. A classifier is then trained on these embeddings to determine whether the given code is correct or not.

- *LLM-as-A-Judge (LLM-AJ)* [37]: These approaches leverage the inherent reasoning and generalization capabilities of LLMs to access code correctness without requiring additional fine-tuning or training. Following the setup of prior work [37], we adopt a zero-shot prompting strategy using the same prompt design, where the model is directly queried regarding the correctness of a given code snippet. The full prompt details are available on our website [36]. In our experiments, we employ zero-shot inference with both the model under study (*In-house LLM-AJ*) (e.g., DEEPSEEK CODER, CODE LLAMA, or MAGICODER) and an external reasoning model o4-mini (*External LLM-AJ*), which serves as an independent verifier to evaluate code correctness.

For a fair comparison, we applied the same neural architecture for the classification models in both post-hoc methods and OPENIA. In each approach, the classifier contains the input, output layers, and two hidden layers with 128 and 64 neurons correspondingly. The training is conducted for 50 epochs with a batch size of 32 and a learning rate of 10^{-3}.

Procedure: To compare OPENIA's performance with the baselines, we extracted the internal state of the last generated token from the last layer of the studied LLM for each query as a representation to assess the correctness of the corresponding generated output. This choice is motivated by the fact that the internal state from the last layer is closest to the model's final decision, making it a logical and effective representation for evaluating the correctness of the generated code. Moreover, the last token is generated based on both the prompt input and the entire generated output, allowing its internal state to encapsulate the most comprehensive information processed by the model. Therefore, this

state is selected for conducting experiments to answer this research question. Note that while this experiment focuses on leveraging the last token's internal state for comparison with baselines, the impacts of other tokens and layers are investigated separately in RQ2.

Note that in our comparative study, while both OPENIA and the post-hoc classification-based methods provide their code quality assessment after the code generation is complete, they are fundamentally different in terms of the input. The post-hoc classification-based methods rely exclusively on the *final output code produced after the generation process*. In contrast, OPENIA gives its assessment based on the code LLMs' internal states which are *the intermediate artifacts produced during the generation process*. This design also enables OPENIA to support early code correctness assessment. For example, by analyzing the internal representations of early generated tokens/layers, correctness assessments could be made even before code generation finishes, allowing proactive interventions.

In this experiment, we compared the performance of OPENIA with the baselines across multiple settings.

- ***Independent-unit code generation*:** This evaluates the correctness of code generated for standalone programming tasks [33, 34].
  - *Cross-benchmark*: The internal representations of a given code LLM obtained while generating code for tasks in one benchmark (i.e., MBPP) are used to train the probing classifier. The classifier is then tested on the internal representations obtained while generating code for tasks in a different benchmark (i.e., HumanEval) with the same code LLM.
  - *Cross-task*: The tasks within a benchmark are split in a 9:1 ratio, where 90% of the tasks were used to extract internal representations for training and validating the probing classifier, and 10% of the remaining tasks are used for testing.
- ***Repo(sitory)-level code generation*:** This evaluates the correctness of code generated for existing projects. Unlike independent-unit code generation, repo-level tasks require LLMs to maintain contextual consistency across multiple code units, making them significantly more challenging [47, 48]. This setting measures how well OPENIA can handle larger, context-dependent codebases.

RQ2. Layer and token selection analysis: This experiment investigates the impact of internal representations from different layers and token positions on OPENIA's performance in assessing code correctness. To answer this research question, we systematically evaluated the performance of OPENIA when it is trained and tested on internal states extracted from:

- Different layers within the code LLM, ranging from shallow to deep layers, to determine how abstraction levels influence correctness prediction.
- Various token positions, including first and last generated token, first and last generated code token, to understand their respective contribution to accuracy.

RQ3: Sensitivity analysis: This experiment examines OPENIA's ability to generalize across programming languages. Specifically, we evaluated whether OPENIA, trained on the internal states extracted from the generation of code in several programming languages, can effectively predict the correctness of code in a completely different language. In addition, we also investigated the performance of OPENIA with code generated for tasks of different difficulty levels.

4.4. Metrics

The task of accessing the correctness of code generated by LLM can be formulated as a binary classification task. Specifically, given a generated code snippet (along with related information, such as its internal representations), OPENIA and the baseline approaches aim to predict whether the code is correct or not. To evaluate the performance of the approaches, we adapted widely used classification metrics: *Accuracy, Precision, Recall*, and *F1-Score*.

Given the potential class imbalance in the dataset, as highlighted in Table 1, where the ratio of correct to incorrect code snippets may vary significantly, it is critical to ensure a fair evaluation across both labels. To address this, we employed weighted metrics, including *Weighted Accuracy*, *Weighted Precision*, *Weighted Recall*, and *Weighted F1-score*. These metrics compare the corresponding result for each label and return the average considering the proportion for each label in the test set.

5. Experimental Results

5.1. Performance Analysis

5.1.1. Performance Comparison

Table 2 and Table 3 show the performance of code correctness assessment for independent-unit code generation and repo-level code generation across the studied code LLMs, DEEPSEEK CODER-1.3B and 6.7B, CODE LLAMA 7B and 13B, and MAGICODER-7B. The evaluation includes LLM-as-a-judge (LLM-AJ) methods using the LLM under study (IN-HOUSE LLM-AJ) and a reasoning LLM o4-mini (EXTERNAL LLM-AJ), post-hoc methods with CodeBERT and CodeT5+, and our approach OPENIA. Overall, OPENIA *demonstrates robust and strong performance across all metrics and evaluation settings, maintaining its top results regardless of model size or task complexity.*

In *Cross-benchmark* (Table 2), although OPENIA lags behind EXTERNAL LLM-AJ, it significantly outperforms IN-HOUSE LLM-AJ and the other baseline methods. For example, OPENIA achieves an Accuracy of 0.69 (for CODE LLAMA-7B), surpassing the the traditional post-hoc classification methods by a relative improvement of **10% to 18%**. In *Cross-task*, OPENIA and EXTERNAL LLM-AJ achieve competitive performance. OPENIA obtains accuracies ranging from 0.73 (for DEEPSEEK CODER-1.3B) to 0.79 (for

Table 2
Correctness assessment performance for ***Independent-unit Code Generation***

Code LLM	Setting		Accuracy	Precision	Recall	F1-Score
DEEPSEEK CODER-1.3B	*Cross-benchmark*	IN-HOUSE LLM-AJ	0.55	0.62	0.55	0.50
		EXTERNAL LLM-AJ	0.93	0.93	0.93	0.93
		CodeBERT	0.50	0.52	0.50	0.45
		CodeT5+	0.47	0.52	0.47	0.34
		LLM-CHECK	0.61	0.63	0.61	0.62
		OPENIA	0.66	0.66	0.66	0.66
	Cross-task	IN-HOUSE LLM-AJ	0.42	0.42	0.42	0.42
		EXTERNAL LLM-AJ	0.86	0.86	0.86	0.86
		CodeBERT	0.64	0.64	0.64	0.63
		CodeT5+	0.66	0.66	0.66	0.66
		LLM-CHECK	0.58	0.58	0.58	0.58
		OPENIA	0.73	0.73	0.73	0.73
DEEPSEEK CODER-6.7B	*Cross-benchmark*	IN-HOUSE LLM-AJ	0.32	0.69	0.32	0.26
		EXTERNAL LLM-AJ	0.94	0.94	0.94	0.94
		CodeBERT	0.35	0.62	0.35	0.35
		CodeT5+	0.66	0.65	0.66	0.66
		LLM-CHECK	0.61	0.60	0.61	0.61
		OPENIA	0.65	0.69	0.65	0.67
	Cross-task	IN-HOUSE LLM-AJ	0.68	0.78	0.68	0.57
		EXTERNAL LLM-AJ	0.75	0.77	0.75	0.75
		CodeBERT	0.64	0.60	0.64	0.61
		CodeT5+	0.69	0.68	0.69	0.68
		LLM-CHECK	0.65	0.63	0.65	0.63
		OPENIA	0.79	0.79	0.79	0.79
CODE LLAMA-7B	*Cross-benchmark*	IN-HOUSE LLM-AJ	0.61	0.37	0.61	0.46
		EXTERNAL LLM-AJ	0.97	0.97	0.97	0.97
		CodeBERT	0.58	0.54	0.58	0.54
		CodeT5+	0.63	0.62	0.63	0.62
		LLM-CHECK	0.59	0.67	0.59	0.60
		OPENIA	0.69	0.68	0.69	0.68
	Cross-task	IN-HOUSE LLM-AJ	0.50	0.25	0.50	0.33
		EXTERNAL LLM-AJ	0.75	0.75	0.75	0.75
		CodeBERT	0.68	0.68	0.68	0.68
		CodeT5+	0.70	0.70	0.70	0.70
		LLM-CHECK	0.66	0.45	0.66	0.53
		OPENIA	0.75	0.76	0.75	0.75
CODE LLAMA-13B	*Cross-benchmark*	IN-HOUSE LLM-AJ	0.41	0.16	0.41	0.23
		EXTERNAL LLM-AJ	0.95	0.95	0.95	0.95
		CodeBERT	0.61	0.61	0.61	0.61
		CodeT5+	0.62	0.62	0.62	0.62
		LLM-CHECK	0.56	0.65	0.56	0.56
		OPENIA	0.64	0.65	0.64	0.64
	Cross-task	IN-HOUSE LLM-AJ	0.38	0.14	0.38	0.21
		EXTERNAL LLM-AJ	0.75	0.75	0.75	0.75
		CodeBERT	0.65	0.66	0.65	0.65
		CodeT5+	0.67	0.67	0.67	0.67
		LLM-CHECK	0.64	0.68	0.64	0.57
		OPENIA	0.78	0.79	0.78	0.78
MAGICODER-7B	*Cross-benchmark*	IN-HOUSE LLM-AJ	0.27	0.07	0.27	0.11
		EXTERNAL LLM-AJ	0.94	0.95	0.94	0.94
		CodeBERT	0.46	0.58	0.46	0.49
		CodeT5+	0.54	0.64	0.54	0.56
		LLM-CHECK	0.60	0.64	0.60	0.53
		OPENIA	0.56	0.67	0.56	0.58
	Cross-task	IN-HOUSE LLM-AJ	0.48	0.73	0.48	0.47
		EXTERNAL LLM-AJ	0.80	0.81	0.80	0.80
		CodeBERT	0.75	0.73	0.75	0.73
		CodeT5+	0.68	0.68	0.68	0.68
		LLM-CHECK	0.67	0.68	0.67	0.67
		OPENIA	0.78	0.78	0.78	0.78

DEEPSEEK CODER-6.7B), significantly exceeding the performance of the post-hoc classification methods by up to **23%**. This superior performance highlights the effectiveness of leveraging the internal representations of the LLM, demonstrating OPENIA's capability to assess the correctness of generated code with high reliability.

For *repo-level code generation* (Table 3), OPENIA also demonstrate impressive performance, consistently surpasses the baseline approaches, including all the studied LLM-AJ methods, post-hoc methods, and LLM-CHECK. For example, OPENIA obtains F1-Scores ranging from 0.73 to 0.83, while the corresponding figures of IN-HOUSE LLM-AJ and the post-hoc methods based on CodeBERT and CodeT5+ are between 0.27 and 0.83. Especially for DEEPSEEK CODER-1.3B, OPENIA achieves an Accuracy of 0.86 with an F1-Score improvement of up to **three times** over the baselines,

Table 3
Correctness assessment performance for ***Repo-level Code Generation***

Code LLM		Accuracy	Precision	Recall	F1-Score
DEEPSEEK CODER-1.3B	IN-HOUSE LLM-AJ	0.24	0.76	0.24	0.27
	EXTERNAL LLM-AJ	0.87	0.87	0.87	0.87
	CodeBERT	0.86	0.78	0.86	0.81
	CodeT5+	0.86	0.81	0.86	0.83
	LLM-CHECK	0.86	0.76	0.86	0.81
	OPENIA	0.86	0.82	0.86	0.83
DEEPSEEK CODER-6.7B	IN-HOUSE LLM-AJ	0.48	0.70	0.48	0.50
	EXTERNAL LLM-AJ	0.70	0.73	0.70	0.71
	CodeBERT	0.71	0.67	0.71	0.68
	CodeT5+	0.73	0.69	0.73	0.69
	LLM-CHECK	0.74	0.54	0.74	0.63
	OPENIA	0.75	0.73	0.75	0.73
CODE LLAMA-7B	IN-HOUSE LLM-AJ	0.51	0.65	0.51	0.54
	EXTERNAL LLM-AJ	0.73	0.76	0.73	0.74
	CodeBERT	0.74	0.68	0.74	0.69
	CodeT5+	0.76	0.71	0.76	0.71
	LLM-CHECK	0.77	0.59	0.77	0.67
	OPENIA	0.81	0.79	0.81	0.79
CODE LLAMA-13B	IN-HOUSE LLM-AJ	0.19	0.04	0.19	0.06
	EXTERNAL LLM-AJ	0.77	0.79	0.77	0.78
	CodeBERT	0.80	0.73	0.80	0.74
	CodeT5+	0.78	0.70	0.78	0.73
	LLM-CHECK	0.80	0.66	0.80	0.73
	OPENIA	0.81	0.77	0.81	0.78
MAGICODER-7B	IN-HOUSE LLM-AJ	0.74	0.64	0.74	0.65
	EXTERNAL LLM-AJ	0.70	0.74	0.70	0.72
	CodeBERT	0.73	0.68	0.73	0.69
	CodeT5+	0.72	0.67	0.72	0.69
	LLM-CHECK	0.75	0.56	0.75	0.64
	OPENIA	0.77	0.73	0.77	0.73

while for CODE LLAMA-7B, OPENIA's Accuracy is 10% relatively better than that of EXTERNAL LLM-AJ. These findings underscore the advantage of leveraging internal representations, making OPENIA more reliable in assessing the correctness of the real-world and complex repo-level code generated by the LLMs.

In our experiments, for IN-HOUSE LLM-AJ with the studied LLMs (i.e., DEEPSEEK CODER, CODE LLAMA, or MAGICODER), we observe an imbalance in their ability to handle positive and negative cases. While these models can achieve competitive Precision, they struggle with Recall. For example, DEEPSEEK CODER-6.7B achieves Precision scores of 0.69 and 0.70 in the independent-unit and repo-level code generation settings, respectively, which is comparable to the other approaches. However, its Recall is up to about two times lower than the others. The reason is that these models tends to be over-confident in the correctness of its generated code, frequently labeling its generated code as "correct" even when it is actually incorrect.

In practice, LLM-AJ requires no additional training but exhibits variable performance across settings, as shown by the drop in LLM-AJ's accuracy between the independent-unit and repo-level code generation scenarios. Meanwhile, OPENIA delivers consistently robust performance by leveraging a lightweight classifier trained on internal states. To ensure broad generalization, we evaluated OPENIA across cross-benchmark, cross-task, and repository-level scenarios, each featuring distinct data distributions. In every case, OPENIA maintained high accuracy, precision, recall, and F1-Score, demonstrating its resilience to domain shifts and varying code structures. For a practical application, practitioners can pre-train OPENIA on curated data from prior tasks or publicly available datasets. During deployment, new data can be incrementally collected and used to periodically update OPENIA, thereby improving its performance for task-specific and evolving use cases.

The traditional post-hoct approaches, which predict code correctness using the code semantics encoded by pre-trained models such as CodeBERT and CodeT5/CodeT5+, demonstrate a quite strong performance. Despite their strengths, these baselines are outperformed by OPENIA, as evidenced by consistently better performance across all models in Table 2 and Table 3. For repo-level code generation, OPENIA

achieves an F1-Score of 0.73 with DEEPSEEK CODER-6.7B, compared to 0.69 for CodeT5+. Especially for CODE LLAMA-7B, the margin of improvement is even more significant, with OPENIA attaining F1-Scores of 0.79, respectively, compared to 0.71 and 0.69 for the classifier based on CodeT5+ and CodeBERT. These results suggest that relying on pre-trained models to encode generated code, while effective to an extent, is insufficient for capturing the nuanced, dynamic, and process-oriented aspects of code correctness.

Unlike these black-box (closed-box) approaches that rely solely on final outputs, both OPENIA and LLM-CHECK are white-box (open-box) approaches that examines the LLMs' internal workings to make predictions. LLM-Check combines an Eigen-analysis of hidden states and attention maps with token-level uncertainty quantification to flag hallucinations without any additional training [40]. While this zero-training design makes LLM-Check immediately deployable, its performance on code-quality assessment remains lower than that of OPENIA up to 40%. By contrast, OPENIA analyzes the intermediate states of the LLM during code generation and trains a lightweight classifier on these in-process signals. This approach uncovers hidden patterns associated with the generated code, mitigating the overconfidence often seen in LLM-AJ approach and providing a more nuanced and reliable evaluation. Moreover, the internal representations utilized by OPENIA not only encode the code features as CodeBERT and CodeT5+ but also capture the model's *reasoning process*. This reasoning process information reflects not just dataset-specific patterns but also fundamental programming underlying programming principles. OPENIA can dynamically adapt to varying requirements, programming constructs, and output expectations across different tasks and benchmarks, enhancing its generality and robustness. This adaptability enhances its generality and robustness, enabling it to consistently outperform both post-hoc classifiers and LLM-CHECK in our experiments across all experimental settings. Additionally, by leveraging these *in-process* signals, OPENIA can provide a unique advantage: *it can detect potential issues and assess correctness as the code is being generated, enabling real-time quality assurance*.

Furthermore, OPENIA *also offers a promising alternative approach for benchmarking LLMs in scenarios where ground-truth test cases are unavailable or costly to produce*. Indeed, in our experiments, we found that code samples labeled "correct" by OPENIA passed actual test cases at rates **50–250%** higher than those labeled "incorrect". Moreover, for each setting, we measured the performance of every studied code LLM in *pass@1*, as well as the proportions of generated code samples labeled "correct" ($\mathcal{P}$) by OPENIA. We observed strong positive correlations with the Pearson correlation coefficients between *pass@1* and $\mathcal{P}$ are 0.99, 0.73, and 0.97 for Cross-benchmark, Cross-task, and Repo-level settings respectively. These findings demonstrate that models ranked higher by OPENIA tend to achieve higher test-pass rates in practice. As a result, developers can generate code for identical tasks from different LLMs and use OPENIA to estimate each model's relative correctness. The LLM that produces a higher proportion of predicted-correct outputs can be deemed superior. This opens up a practical and test-free solution for LLM comparison in settings where constructing tests is costly.

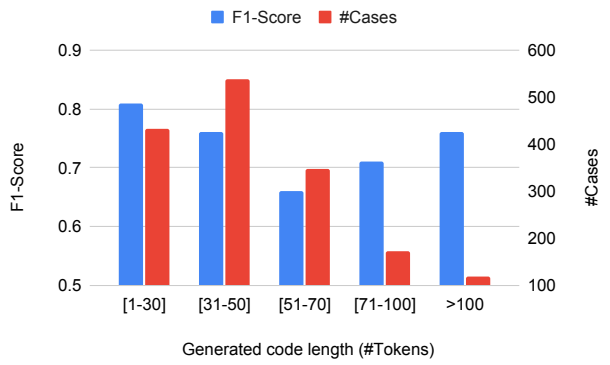


Figure 4: Impact of the length of the generated code on OPENIA's performance

5.1.2. Code Length Analysis

Figure 4 illustrates that OPENIA achieves its best performance on shorter generated code. Specifically, for the code snippets ranging from 1 to 30 tokens, OPENIA accurately predicts code correctness with an F1-Score of 0.81. This strong performance is attributed to the simplicity of short code snippets, which typically contain few dependencies. Thus, the semantic and syntactic features of the code can be effectively encoded within the LLM's internal states, thereby facilitating accurate predictions. However, OPENIA's F1-score decreases as the code length increases, reaching its lowest point (0.66) for medium-length code (51-70 tokens). The decline in performance is likely due to the increased complexity and dependencies in longer code, which makes it more challenging for the LLM's internal representations to encode all critical information.

Interestingly, OPENIA exhibits performance recovery for longer code exceeding 70 tokens, despite the challenges posed by increased complexity. This improvement suggests that OPENIA can effectively utilize the richer contextual information and redundancy available in the internal states of longer code to reinforce key patterns and enhance correctness predictions. These results highlight the adaptability of OPENIA in leveraging the LLM's representations across varying lengths.

5.1.3. Error Analysis

Table 4 presents a detailed evaluation of OPENIA's performance across different error categories. To compute F1-Score for each error type, we first categorize the generated code snippets into distinct error types according to their ground truth testing results. For each error category, we measure how accurately OPENIA identified the corresponding code snippets as incorrect. In this table, the *Correct* row represents OPENIA's performance in accurately recognizing

Table 4
OPENIA's performance on different errors

Category	*F1-Score*	*#Cases*
Correct	0.75	644
Timed out	0.89	10
AssertionError	0.82	825
Not defined variables/attributes	0.86	36
Unsupported operands	0.82	13
Index out of range	0.73	19
Type error	0.83	33
Others	0.85	60

Table 5
Performance of Code LLMs when applying *Correctness-Guided Generation (CG^2)* using OPENIA (pass@1)

		Without CG^2	**With CG^2**
DEEPSEEK CODER-6.7B	*Cross-benchmark*	0.75	0.77
	Cross-task	0.63	0.67
	Repo-level	0.26	0.28
CODE LLAMA-7B	*Cross-benchmark*	0.60	0.61
	Cross-task	0.51	0.57
	Repo-level	0.22	0.24
MAGICODER-7B	*Cross-benchmark*	0.74	0.76
	Cross-task	0.69	0.73
	Repo-level	0.24	0.27

correct code, while the remaining rows reports OPENIA's performance in detecting incorrect code in each error type.

Overall, OPENIA demonstrates higher accuracy in detecting incorrect code compared to the correct one. Specifically, OPENIA's F1-Score for identifying correct code is 0.75, whereas its performance in detecting incorrect code reaches 0.82. This is reasonable because identifying incorrectness often requires detecting only a single error-indicating signal/pattern, making it relatively easier to classify a code snippet as incorrect. In contrast, determining correctness is inherently more challenging, as it necessitates verifying multiple aspects, including logic, structure, and functionality, to ensure the absence of errors.

Among the error types, OPENIA exhibits the lowest performance in detecting incorrect code caused by *index out of range* errors. Specifically, its F1-Score for identifying incorrect code of this error is 0.73, which is 12% lower than its average performance of detecting incorrect code. Unlike *static* errors, such as undefined variables or type errors, that are inherently present in the code and can be directly captured in the LLM's internal states, index out-of-range errors are not always explicitly encoded in the internal representations of the model. This is because such errors often depend on *dynamic* factors like runtime conditions or specific input values. Therefore, effectively detecting incorrect code of this error could require a deeper understanding of program execution and data dependencies.

Table 4 shows our analysis on various error categories for evaluation purposes. While OPENIA outputs a binary classification, indicating whether the generated code is correct or not, it does not provide detailed diagnostics such as error types or bug locations. As a result, in cases where the code is predicted to be incorrect, developers still need to rely on complementary analysis techniques, such as static analysis, dynamic testing, or multi-agent frameworks, to identify the root cause and guide repair. Despite this limitation, OPENIA offers practical benefits in several scenarios. First, it can be integrated into the code generation pipeline to proactively filter promising candidates and discard low-quality ones, thereby reducing the risk of presenting incorrect code to developers and minimizing debugging efforts. Second, OPENIA can serve as a lightweight examiner to determine whether more computationally expensive diagnostic processes are necessary. For future work, we plan to extend OPENIA by integrating bug localization and repair phases to enhance its practical usages.

5.1.4. Potential in Improving LLM-gen. Code's Quality

To demonstrate the potential of OPENIA in proactively detecting and preventing bugs before they reach developers, we introduce a *Correctness-Guided Code Generation (CG^2)* pipeline. For each programming task, a LLM is employed to generate several candidates. OPENIA is then used to assign the correctness score to each candidate, and the highest-scoring one is selected and presented to developers. We compare this pipeline against the standard generation process using the *pass@1* metric [33]. Note that, because OPENIA extracts internal states during generation and feeds them to a lightweight probing classifier with only two hidden layers, the additional time overhead is marginal (more details in Sec. 5.4). Table 5 shows the performance (in pass@1) of the code LLMs, with and without CG^2, across different evaluation settings. As seen, OPENIA is effective in guiding the selection of high-quality code, as demonstrated by the consistent improvements of the LLMs with CG2 over the LLMs without CG2. By selecting the highest-scoring candidates predicted by OPENIA, the average *pass@1* of the code LLMs across different settings improves by about 7%.

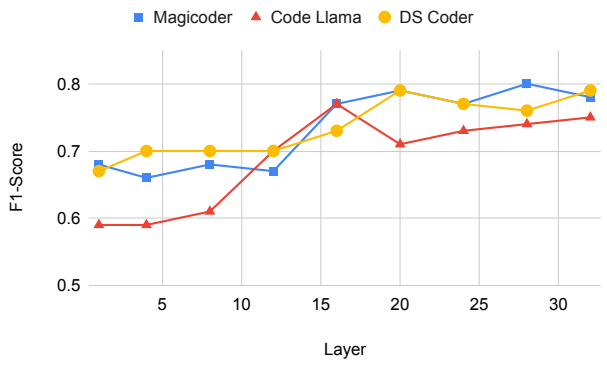
We also evaluate a filtering variant in which any solution labeled "incorrect" by OPENIA is proactively discarded before developer review. Compared to the standard code generation process, this filtering pipeline improves the average rates of passed tests by 6%–86%. Especially, in *Repo-level* setting, the improvements are at least 53%.

Overall, *these results highlight the potential of incorporating* OPENIA *into the code generation pipeline to enhance the quality of the generated code*.

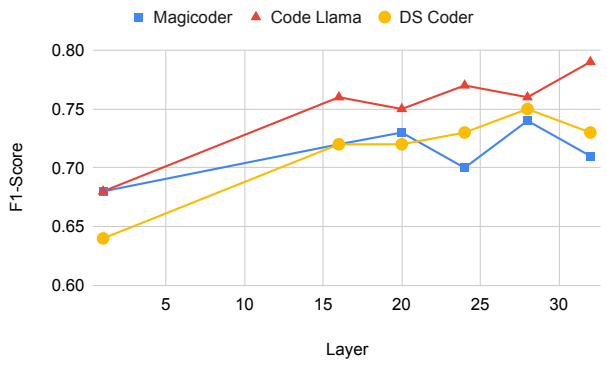
5.2. Layer and Token Selection Analysis

5.2.1. Layer Selection Analysis

Figure 5 shows the performance of OPENIA in correctness assessment when using hidden states from different layers of three code LLMs: MAGICODER-7B, CODE LLAMA-7B, and DEEPSEEK CODER-6.7B. In general, *the representations from the middle layers of the models enable* OPENIA *to achieve optimal performance for both independent-unit and repo-level code generation tasks*. For independent-unit



(a) Independent-unit code generation



(b) Repo-level code generation

Figure 5: Impact of layer selection on OPENIA's performance

code generation (Fig. 5a), OPENIA obtains its best performance by leveraging the representations from layer 28 of MAGICODER, layer 16 of CODE LLAMA, and layer 20 of DEEPSEEK CODER. Similarly, for repo-level code generation (Figure 5b), OPENIA attains its highest F1-Score using the representations from layer 28 for both MAGICODER and DEEPSEEK CODER. For CODE LLAMA, while OPENIA's peak performance is achieved with the representations from the last layer (layer 32), the middle layers (layers 16-28) still offer consistently strong results. These findings suggest that the middle layers in the LLMs typically capture a balance between the local context focused by the shallow layers and the global context encoded by the deep layers, making them optimal for assessing code correctness.

The shallow layers primarily focus on syntax and token-level details, capturing the immediate relationships between adjacent tokens (local context). While such information is useful for understanding the basic structure, it is insufficient for determining the correctness of the generated code. Consequently, leveraging the representations from the shallow layers often leads to low performance of OPENIA. For example, when using the representations from the first layer of CODE LLAMA, OPENIA achieves an F1-Score of 0.59, which is 30% lower than its highest score.

In contrast, the deep layers often capture broader logical flow and long-range dependencies between code tokens (global context). The representations from these layers provide richer semantic information, improving OPENIA's ability to assess code correctness. However, the deeper the layers, the closer they are to the model's final output, making the representations at the deeper layers more task-specific. Leveraging the representations from too deep layers could make OPENIA to be overfitting with the training data and potentially hinder its generality. For instance, OPENIA observes a slight decline in performance when using the representations from the last layer of several models.

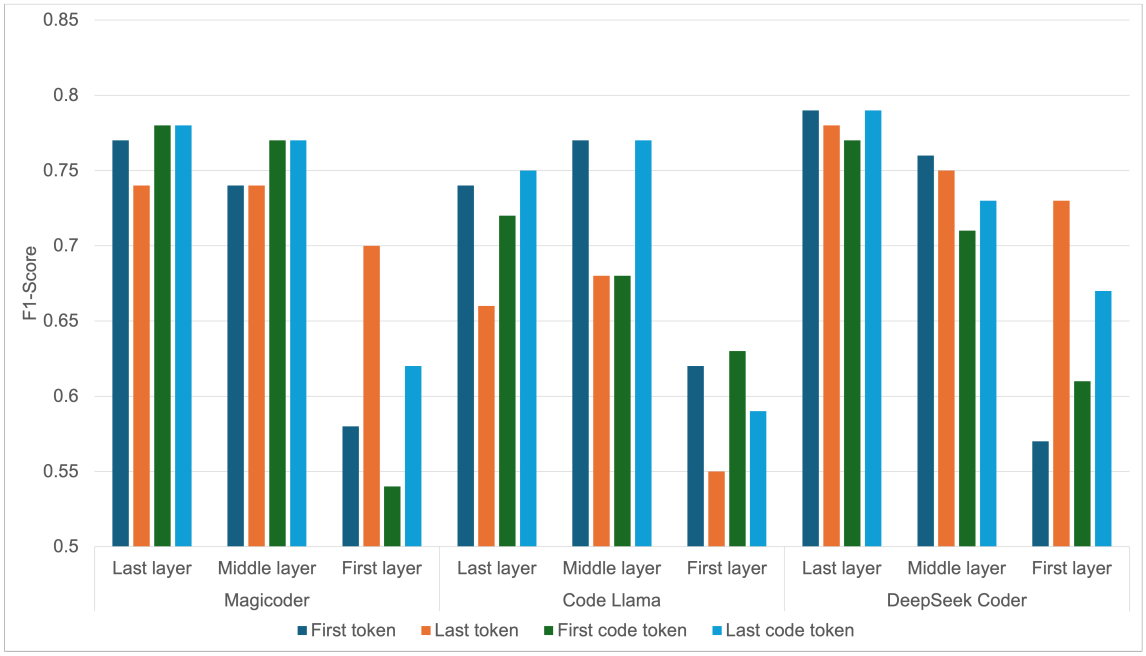
The middle layers strike a balance between the shallow and the deep layers, providing rich representations that are often suitable for downstream tasks such as code correctness assessment. However, the specific layer at which the performance of OPENIA peaks or dips can vary depending on the model's architecture and its pre-training objectives. This suggests that static layer selection may not be ideal, as there is no one-size-fits-all approach to selecting the best layers for OPENIA. Instead, OPENIA *could benefit from a dynamic approach that adapts layer selection based on the models and task requirements*.

5.2.2. Token Selection Analysis

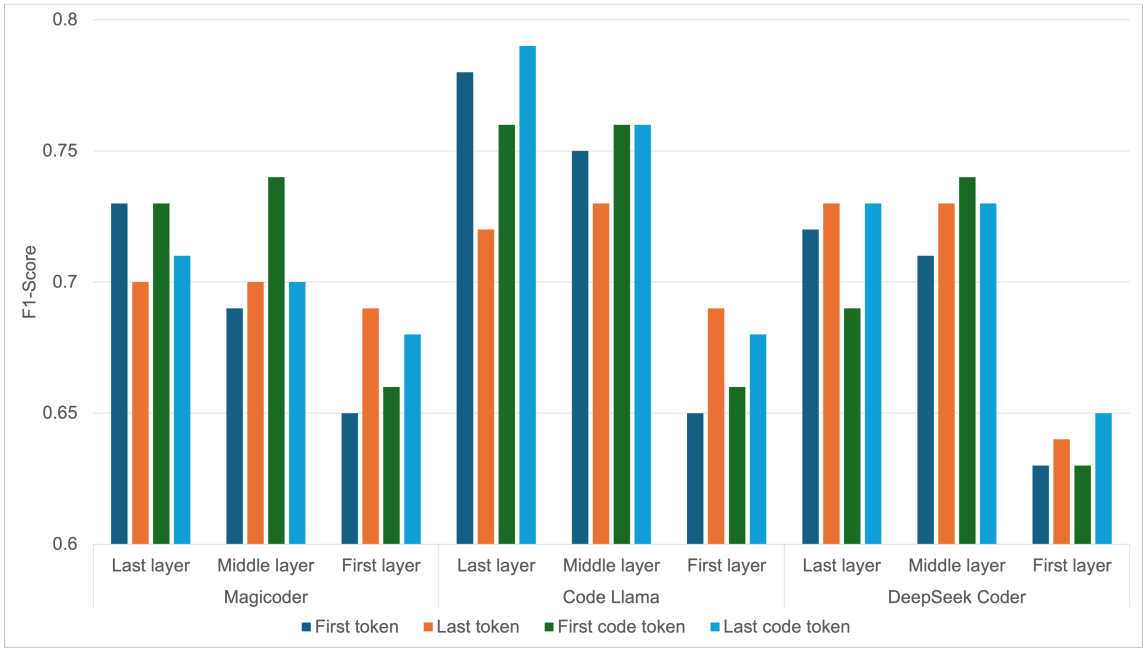
Figure 6 shows the performance of OPENIA when using representations of different tokens from different layers of three models, MAGICODER-7B, CODE LLAMA-7B, and DEEPSEEK CODER-6.7B. In all three models, the first layer corresponds to layer 1, the middle layer is layer 16, and the last layer is layer 32. For each layer, we extract and evaluate the performance of OPENIA with the representations of tokens at four specific positions: *first token* and *last token*, which represent the starting and ending points of the entire answer generated by the LLMs, and *first code token* and *last code token*, which correspond to the starting and ending points of the code segment within the LLMs' response.

In general, *the deeper layer, the more stable* OPENIA*'s performance is across the internal representations from different tokens; conversely, the shallower layer, the more sensitive* OPENIA *becomes to variations in token selection.* Specifically, when using the representations from the last layers, OPENIA consistently achieves the highest F1-Score with the representations of the ***last code token***. Furthermore, its performance across different token representations from the last layers remains relatively stable. In contrast, when using representations from the first layer, there is a noticeable performance gap between different tokens. For example, in assessing the correctness of code generated by MAGICODER for independent-unit tasks, (Fig. 6a), OPENIA achieves an F1-Score of 0.7 with the *last token* representations from the first layer, which is 30% higher than its performance with the *first code token* representations.

Indeed, the first layer operates directly on raw token embeddings, and the interactions/relationships between tokens have not yet been captured. Due to the lack of contextualization, different tokens could convey varying levels of information, leading to different impacts on OPENIA's performance. This explains why OPENIA is highly sensitive to the token selection when using representations from the



(a) Independent-unit code generation



(b) Repo-level code generation

Figure 6: Impact of token selection on OPENIA's performance

first layer of the models. Meanwhile, by the time tokens reach the last layer, their representations have been enriched with contextualized information, incorporating the relationships between tokens across the sequence. As a result, representations from the last layer are more robust and less dependent on the specific token chosen, leading to stable performance of OPENIA across different tokens.

However, the impact of token selection on OPENIA's performance varies depending on the model and layer. For instance, MAGICODER exhibits less sensitivity to token selection, while CODE LLAMA is highly affected. For example, with the representations from the last layer of MAGICODER, OPENIA's F1-score in predicting the code correctness for independent-unit code generation ranges from 0.74 to 0.78. However, these figures for CODE LLAMA vary more widely, 0.66–0.75. This highlights the importance of carefully selecting suitable tokens to extract representations to obtain the best performance of OPENIA. Similar to layer selection, *token selection should be dynamically adapted to align with the specific architecture and characteristics of the model*.

Table 6
Correctness assessment performance in F1-Score of the approaches across different languages

Targeted language	CodeBERT	CodeT5+	OPENIA
CPP	0.65	0.67	0.82
C Sharp	0.74	0.75	0.80
Java	0.59	0.64	0.68
JavaScript	0.66	0.80	0.86
PHP	0.54	0.65	0.71
Python	0.37	0.50	0.67
Shell script	0.58	0.58	0.64
TypeScript	0.72	0.82	0.88
MIX	0.66	0.67	0.72
AVERAGE	0.61	0.68	0.75

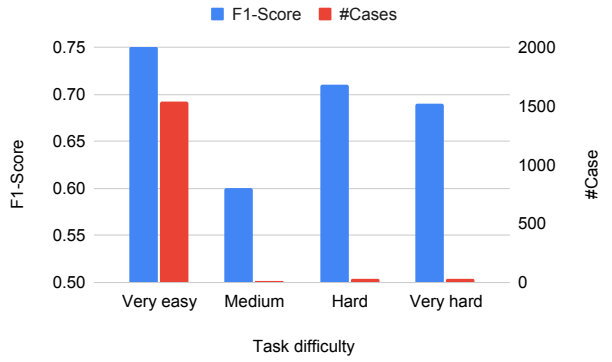


Figure 7: OPENIA's performance on code generated for tasks of different difficulty levels

5.3. Sensitivity Analysis

5.3.1. Programming Language Analysis

Table 6 shows the generalization performance of OPENIA and post-hoc classification-based approaches in assessing the code correctness across different programming languages. This experiment is conducted on the multi-language version of HumanEval benchmark. Specifically, for each task in each programming language, DEEPSEEK CODER-6.7B is employed to generate 10 candidate solutions. To evaluate cross-language generalization, we select one target programming language for testing, and use the generated code snippets (for post-hoc classifier) or internal states (for OPENIA) from the remaining languages for training. In addition, the row labeled "MIX" in the table corresponds to a setting where all generated code across all languages is combined and randomly split into training and testing sets using a 9:1 ratio. This setting enables to evaluate performance of approaches on mixed language data.

OPENIA *consistently outperforms the traditional post-hoc methods that rely on CodeBERT or CodeT5+ to encode the final LLM-generated code*. Across all the studied programming languages, OPENIA achieves an average F1-Score of 0.75, which is 12% higher than CodeT5+ and 23% higher than CodeBERT. Notably, for Python, OPENIA surpasses CodeT5+ and CodeBERT by 35% and 81%, respectively. These results indicate that the internal representations of the LLMs can effectively capture not only the code semantics but also the fundamental programming principles encoded in the code generation process. By levering these representations, OPENIA can generalize its learned knowledge to better detect code correctness across different languages.

Furthermore, all the approaches achieve their highest performance when the target language of the test set is TypeScript and their lowest performance when the target language is Python. For example, OPENIA's F1-Score for predicting the correctness of Python code is 0.67, which is 12% lower than its average score. Meanwhile, for TypeScript code, OPENIA achieves an impressive F1-score of 0.88, exceeding its average score by 17%. A similar trend is observed with CodeBERT and CodeT5+. This is due to Python's inherent characteristics, such as dynamic typing, indentation-based syntax, and Pythonic idioms, which significantly differ from the other languages. These features introduce ambiguity and variability, making it harder for OPENIA and the other approaches to generalize their learned knowledge to Python code. In contrast, TypeScript shares many common characteristics with other languages, such as supporting both static and dynamic typing, verbose and explicit code, and curly braces-based syntax. These shared characteristics allow OPENIA and the other approaches to generalize their learned knowledge more effectively, resulting in higher performance when predicting the correctness of TypeScript code.

5.3.2. Task Difficulty

In this experiment, we investigate how the difficulty levels of the input tasks affect OPENIA's performance. To categorize task difficulty, we first prompt the studied code LLMs to classify each task in the HumanEval dataset into one of five levels: Very easy, Easy, Medium, Hard, and Very hard. The prompt used and the assigned difficulty levels for each task are available on our website [36]. We then analyze the performance of OPENIA in predicting the correctness of the generated code for tasks within each difficulty group.

As shown in Fig. 7, OPENIA obtains higher performance on both easy and hard tasks, while its performance declines for medium-level tasks. Specifically, for easy tasks, OPENIA correctly predicts the correctness of generated code with an F1-score of 0.75, which is 25% higher than its performance on medium-level tasks. These findings suggest that the internal representations of the code LLMs can capture its confidence about its responses. For instance, the code LLMs tend to be more confident when handling easy tasks and less confident with hard ones. This confidence signal can effectively help to assess code correctness, as reflected in OPENIA's high performance on these tasks. However, for medium-level tasks, the confidence signals appear to be less distinct, leading to a decrease in OPENIA's performance.

5.3.3. Prompt Variability

As shown in Table 2 (*Cross-benchmark*), differences in prompt templates between training set (MBPP) and testing

set (HumanEval) could result in a decline in OPENIA's performance, this is not observed for (output-based) post-hoc methods. For DEEPSEEK CODER-6.7B, F1-score is 0.79 under the *Cross-task* setting where the prompt template remains consistent between train and test, while the corresponding figure is 0.67 when the prompt templates in the two sets are different. This is reasonable since LLMs' internal representations inherently encode prompt-specific characteristics, e.g., wording style, instruction phrasing, and contextual emphasis. Consequently, shifts in prompt format induce subtle changes in these latent signals, which in turn affect the features upon which OPENIA's classifier depends.

To mitigate this sensitivity, we plan to augment the training set with diverse paraphrases of each prompt. This approach should stabilize OPENIA's internal-feature space and further bolster its robustness across varied prompt formulations. Preliminary experiments with mixed-prompt training show an improvement with accuracy increasing from 0.68 to 0.72, confirming that prompt augmentation can enhance OPENIA's robustness to prompt variability.

5.3.4. *Training Data Size*

To understand how the amount of training data affects OPENIA's performance, we conducted a controlled experiment using DEEPSEEK CODER-6.7B in the cross-task setting. We held the test set fixed and partitioned the remaining training data into five equal folds. We then trained OPENIA on an incremental sequence of folds, from one fold (20% of the data) up to all five folds (100%).

As shown in Fig. 8, both accuracy and F1-Score rise as the training set increases from one to three folds, climbing from 0.71 (accuracy) and 0.67 (F1-score) at one fold to 0.77 for both metrics at three folds. Beyond three folds, performance gains plateau: four folds yield only marginal improvement, and the full five-fold model performs similarly. This trend indicates that OPENIA quickly learns the key patterns needed to distinguish correct from incorrect code, achieving near-optimal performance with only 60% of the available training data. Adding more data beyond that point produces diminishing returns, suggesting that OPENIA is data-efficient and can be effectively trained with a relatively small corpus. In practice, practitioners can achieve robust correctness assessment without requiring large labeled datasets, reducing annotation cost and accelerating deployment in resource-constrained settings.

5.4. Time Complexity

The time complexity of OPENIA is primarily determined by three factors: preparing training data and extracting internal representations from the studied code LLMs, training the probing classifier, and performing inference.

Preparing the training data for OPENIA and the studied post-hoc approaches on an NVIDIA A6000 GPU with 48GB of VRAM took approximately 86 hours for the entire dataset of 24,890 code units generated by the LLMs. The extraction of internal representations from the studied code LLMs was performed during code generation without affecting the process of the code LLMs. All subsequent

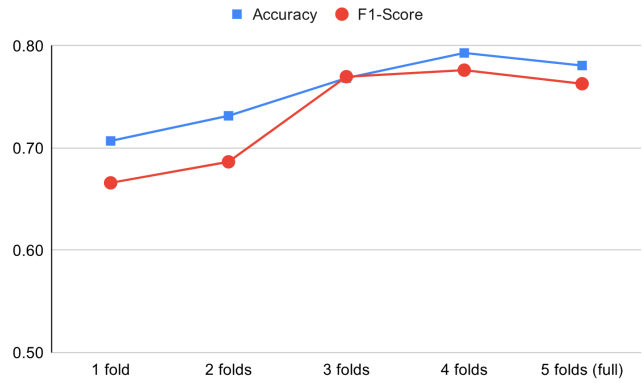


Figure 8: OPENIA's performance on training data size

experiments, including probing classifier training and inference, were conducted on a single NVIDIA P100 GPU with 16GB of VRAM. OPENIA demonstrates high efficiency in both training and inference due to its lightweight probing classifier, which consists of two hidden layers (128 and 64 neurons). The training process takes approximately 1 minute, as there is no embedding computation overhead. In contrast, classifiers built on pre-trained models, such as CodeBERT and CodeT5+, require significantly more time, taking about 83 minutes to train, and the embedding time took about 98% of this amount of time. OPENIA's inference process is extremely fast, requiring only 0.6 milliseconds per generated code unit. This highlights the feasibility of OPENIA for real-time, in-process assessment of code correctness during LLM generation, making it well-suited for practical deployment scenarios such as in interactive and real-time coding environments.

5.5. Threats to Validity

The main threats to the validity of our work consist of internal, construct, and external threats.

Internal Validity. Threats to internal validity include the hyperparameter selection for both OPENIA and the baselines. To mitigate this threat, we systematically explored various hyperparameter configurations for OPENIA and evaluated its performance under varied conditions (as detailed in Sec. 5). For baseline methods, we reused implementations and settings from the original papers, such as those for CodeBERT and CodeT5+ [44, 46], ensuring fair comparison. Additionally, potential threats could arise from the procedures used to extract the internal representations of LLMs and their correlation with code correctness. To address this, we utilized the widely used Pytorch libraries to extract and interpret the intermediate states of LLMs during code generation. These methods were validated on diverse benchmarks and cross-verified using established practices in the field. To further minimize the risk of errors in implementation, our codebase has undergone extensive internal review, and we have made it publicly available [36].

Construct Validity. Threats to construct validity relate to the appropriateness of our evaluation metrics and procedures. We employed widely recognized metrics, including *Accuracy*, *Precision*, *Recall*, and *F1-Score*, to measure the performance of the approaches, as these are standard benchmarks in evaluating classification models. Additionally, we designed experiments to include both standalone and repository-level code generation tasks, ensuring comprehensive evaluations. However, our evaluation primarily relies on automated metrics and does not yet incorporate human assessments. While automated metrics provide objectivity and scalability, human evaluation could offer deeper insights into the practical utility of OPENIA in real-world coding scenarios. We plan to include such assessments in future work, engaging experienced developers to evaluate the generated code based on factors like functionality, readability, and adherence to programming standards. Also, a potential threat is our procedure labeling the correctness of code generated by the studied code LLMs, particularly the "correct/passed" label, due to the potential low-quality tests. To mitigate this threat, we utilized the widely-used benchmarks [33, 34, 35] with adequate test sets. Another potential limitation is that our controlled experimental setup may not fully reflect real-world software development environments. To address this, we evaluated OPENIA on multiple representative benchmarks, covering diverse programming constructs and code generation requirements.

External Validity. Threats to external validity concern the generalizability of our results to different LLMs and application domains. To mitigate this, we selected three widely used and representative open-source code LLMs: DEEPSEEK CODER, CODE LLAMA, and MAGICODER [16, 17, 18]. These models have demonstrated strong performance across diverse code-related tasks, enhancing the applicability of our findings within the code generation domain. A further potential threat arises from our focus primarily on models with parameter sizes up to 7B, due to hardware constraints. While this ensures feasibility within our experimental setup, it may limit the applicability of our conclusions to larger-scale models. As part of future work, we plan to investigate the scalability of OPENIA to larger LLMs and expand our experiments to a broader range of programming languages and codebases to enhance the generalizability of our approach. One potential threat to the robustness of OPENIA lies in the variability of input prompts. However, this threat can be partially mitigated by our evaluation across multiple benchmarks, i.e., HumanEval, MBPP, and DevEval, which exhibits diverse prompt styles and task descriptions. The consistent performance of OPENIA across these diverse benchmarks indicates a certain degree of robustness to prompt variation. Nonetheless, it remains important to systematically evaluate prompt sensitivity and we plan to investigate its impact on OPENIA's performance in future work.

6. Related Work

LLM-based Code Generation. Code generation is an essential task in software development, aimed at assisting programmers by suggesting subsequent code units based on the current context [49, 50, 51, 52, 53]. Recently, *Large Language Models for code (Code LLMs)* [17, 16, 54, 55] such as Codedex, CODE LLAMA, and DEEPSEEK CODER have emerged as promising tools for code generation/completion, offering the potential to automate repetitive tasks and increase developer productivity. These models, trained on massive datasets and equipped with billions of parameters, have demonstrated remarkable performance in code generation and completion [56, 57, 58]. Code LLMs have been deployed as auto-completion plugins, such as Github Copilot and CodeGeeX2 [59] in modern IDEs, and successfully streamline real-world software development activities to a certain degree [22, 60, 61, 62]. Beyond standard code generation and completion, repo(sitory)-level code generation adds another layer of complexity as it requires integrating repository-specific elements such as user-defined APIs, inter-module dependencies, and project-specific code conventions [48, 61, 63, 60]. To address the challenges of repo-level code completion, recent approaches have applied the Retrieval-Augmented-Generation (RAG) strategy. By leveraging repository-specific knowledge, RAG-based methods enhance the capabilities of Code LLMs, significantly improving their performance in completing repo-level code tasks [48, 47, 63, 64, 65].

Quality Assurance for LLM-generated Code. The quality of code generated by LLMs has garnered significant attention due to the increasing reliance on AI-generated code in real-world applications. Several empirical studies have investigated bugs and security issues in LLM-generated code, highlighting the potential risks associated with their use [14, 66, 67, 68, 69]. These studies provide a detailed analysis of common flaws and vulnerabilities, emphasizing the need for robust evaluation and mitigation strategies.

Hallucinations, where LLMs produce plausible yet incorrect or non-functional code, are another prominent issue in code generation [70]. Understanding and addressing these hallucinations is essential for ensuring the reliability of LLMs in software development. Furthermore, research has shown that AI-generated code often contains vulnerabilities, necessitating a deeper investigation into its safety implications [71]. For instance, comparative studies have assessed the security vulnerabilities of ChatGPT-generated code against traditional sources like StackOverflow, revealing notable differences in the safety and correctness of the generated code [72]. Efforts have also been made to analyze the cognitive processes of LLMs during code generation. A study explored whether LLMs pay attention to code structures in a manner similar to human programmers, providing insights into their decision-making and error tendencies [73]. Uncertainty analysis for LLMs has been proposed to measure their confidence in generated outputs, which could aid in identifying potentially unreliable code [74].

To enhance the reliability of LLM-generated code, researchers have introduced methods like "slow-thinking," which instructs LLMs to perform step-by-step analyses of code functionality to detect errors more effectively. Multi-agent frameworks have also been developed to secure code generation using static analysis and fuzz testing, such as AutoSafeCoder [75]. Similarly, LLMSecGuard combines static code analyzers with LLMs to provide enhanced code security, leveraging the strengths of both approaches to mitigate vulnerabilities and ensure code robustness [76].

Bug/Vulnerability Detection in Code. Detecting bugs and vulnerabilities in code is a critical aspect of software quality assurance, aiming to identify potential issues before they lead to security breaches or system failures. Various methods have been proposed to assess the vulnerability of code components, such as files, functions, methods, or even individual lines of code [77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89]. Devign [81] employs a graph-based approach leveraging the Code Property Graph (CPG) to predict vulnerabilities effectively. Similarly, tools like VulDeePecker [79] and SySeVR [90] focus on detecting vulnerabilities at the slice level, enabling more fine-grained identification. To enhance the granularity of vulnerability detection, graph-based models have also been developed. IVDetect [91] applies a graph-based neural network to detect vulnerabilities at the function level while using interpretable models to pinpoint vulnerable statements within suspicious functions. Similarly, VELVET [92] adopts graph-based techniques to detect vulnerabilities directly at the statement level.

More recent approaches integrate pre-trained models like CodeBERT [44] and CodeT5 [45], which have demonstrated significant effectiveness in both function-level and statement-level vulnerability detection tasks. For example, LineVul [93] and LineVD [94] leverage CodeBERT to encode code representations, outperforming traditional methods like IVDetect in identifying vulnerabilities at finer levels of granularity. The success of these pre-trained models is largely attributed to their ability to capture rich contextual information and encode both syntactic and semantic characteristics of code efficiently. As a result, models like CodeBERT and CodeT5 have established themselves as powerful baselines for advancing the field of bug and vulnerability detection, driving progress toward more accurate and effective software quality assurance tools.

Analyzing LLM Internal States in Hallucination Detection and Mitigation. Hallucinations, where LLMs generate plausible but inaccurate or false information, remain a critical challenge in the deployment of these models for real-world applications [19, 20]. TrustLLM [95] provides a comprehensive study on trustworthiness in LLMs, covering multiple dimensions such as trust principles, benchmarks, and evaluation methods for mainstream LLMs. Recently, several studies have introduced white-box/open-box approaches in detecting and mitigating hallucination of general-purpose LLMs, which require analyzing the internal states of LLMs to uncover the root causes and enhance their reliability [96, 25]. Azaria *et al.* [97] introduce a classifier trained on the hidden layer activations of LLMs that can predict the probability of a statement being truthful. Another study by Ji *et al.* [37] explores the internal mechanisms across diverse Natural Language Generation (NLG) tasks, analyzing over 700 datasets to uncover patterns in LLM behavior. INSIDE [39] leverages the dense semantic information within LLMs' internal states to detect hallucinations. FactoScope [98] further investigates factual discernment by measuring the inner states of LLMs, providing insights into their capacity to differentiate truth from hallucination.

While our work and prior studies [37, 39, 97, 99] similarly leverage internal representations of LLMs to detect hallucination, OPENIA fundamentally differs in both *task objective* and *methodological approach*. These exiting studies [37, 39, 97, 99] primarily focus on hallucination detection in NLG, where outputs are typically unstructured and correctness is subjective, relying on human judgment or alignment with external factual knowledge. In contrast, OPENIA targets code generation, in which correctness is objectively defined through syntax, logic, and runtime behavior, as can be verified by functional test cases. Moreover, due to the nature of NLG tasks, prior works mainly investigate semantic consistency or factual alignment within model responses. Azaria *et al.* [97] detect hallucinations by training a classifier on the internal representations of both true and false factual statements. INSIDE [39] measures the semantic diversity and consistency across multiple responses to the same question to assess the risk of hallucination. However, these techniques are not directly applicable to code. Unlike natural language, where factual inconsistency often indicates hallucination, a programming task may have multiple correct solutions that differ in implementation details or algorithms. As a result, inconsistency across generated responses does not necessarily imply incorrectness. This key difference highlights the need for a specific approach for code generation assessment like OPENIA.

Beyond analyzing internal states, alternative methods have been proposed to enhance the truthfulness and reliability of LLM outputs. LLMGuardrail [100] integrates causal analysis and adversarial learning to develop unbiased steering representations for improving output quality. Look-Back [38] uses attention maps to identify and mitigate contextual hallucinations. Fact-checking through token-level uncertainty quantification [101] provides a complementary approach for verifying the factual accuracy of LLM-generated outputs. Inference-Time Intervention (ITI) [102] focuses on enhancing LLM truthfulness by dynamically modifying model activations during inference.

In general, these white-box approaches [38, 39, 97, 37], leverage internal neural signals of LLMs such as hidden layers, attention weights, or token embeddings, etc. to explore the *reasoning process* of LLMs during inference. This reflects a model-centric perspective, which seeks to understand how the model internally "thinks" and makes decisions, even when its reasoning is not explicitly expressed in the output. Meanwhile, chain-of-thought (CoT) prompting

is an output-centric technique that encourage the model to produce step-by-step reasoning as part of its response [103]. CoT treats this verbalized reasoning as a proxy for the model's internal thought process. While both approaches aim to uncover how LLMs arrive at their outputs, they differ in whether reasoning is inferred from internal representations or observed directly in the generated response.

7. Conclusion

This work introduces OPENIA, a novel framework that leverages the internal representations of code LLMs to assess the correctness of generated code. Unlike traditional black-box approaches, OPENIA adopts a white-box methodology, systematically analyzing the intermediate states of specialized code LLMs such as DEEPSEEK CODER, CODE LLAMA, and MAGICODER. Our empirical findings demonstrate that these internal signals capture latent information, including fundamental programming principles, which strongly correlate with the reliability of the generated code. OPENIA not only provides a more nuanced and reliable evaluation of code correctness but also exhibits strong adaptability and robustness across diverse benchmarks and tasks. By unlocking the potential of in-process signals, OPENIA bridges the gap between code generation and quality assurance, offering a proactive approach to enhancing the reliability of LLM-generated code. Our framework sets the foundation for future research into leveraging the latent capabilities of LLMs to improve AI-driven software development.

Acknowledgement

This research is supported by Vietnam National Foundation for Science and Technology Development (NAFOSTED) under grant number 102.03-2023.14. This research is also partly supported by OpenAI's Researcher Access Program.

References

[1] J. T. Liang, C. Yang, B. A. Myers, A large-scale survey on the usability of ai programming assistants: Successes and challenges, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 1–13.

[2] J. Jiang, F. Wang, J. Shen, S. Kim, S. Kim, A survey on large language models for code generation, arXiv preprint arXiv:2406.00515.

[3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, ACM Transactions on Software Engineering and Methodology 33 (8) (2024) 1–79.

[4] M. R. Lyu, B. Ray, A. Roychoudhury, S. H. Tan, P. Thongtanunam, Automatic programming: Large language models and beyond, ACM Transactions on Software Engineering and Methodology.

[5] K. Jesse, T. Ahmed, P. T. Devanbu, E. Morgan, Large language models and simple, stupid bugs, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR), IEEE, 2023, pp. 563–575.

[6] F. Tambon, A. M. Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, G. Antoniol, Bugs in large language models generated code, arXiv preprint arXiv:2403.08937.

[7] Z. Liu, Y. Tang, X. Luo, Y. Zhou, L. F. Zhang, No need to lift a finger anymore? assessing the quality of code generation by chatgpt, IEEE Transactions on Software Engineering.

[8] J. Liu, C. S. Xia, Y. Wang, L. Zhang, Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, Advances in Neural Information Processing Systems 36.

[9] H. Krasner, The cost of poor software quality in the us: a 2022 report, Consortium for Information and Software Quality (CISQ).

[10] N. Perry, M. Srivastava, D. Kumar, D. Boneh, Do users write more insecure code with ai assistants?, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 2023, pp. 2785–2799.

[11] R. Khoury, A. R. Avila, J. Brunelle, B. M. Camara, How secure is code generated by chatgpt?, in: 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2023, pp. 2445–2451.

[12] J. He, M. Vechev, Large language models for code: Security hardening and adversarial testing, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 2023, pp. 1865–1879.

[13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? assessing the security of github copilot's code contributions, in: 2022 IEEE Symposium on Security and Privacy (SP), IEEE, 2022, pp. 754–768.

[14] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, B. Dolan-Gavitt, Lost at c: A user study on the security implications of large language model code assistants, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 2205–2222.

[15] P. Vaithilingam, T. Zhang, E. L. Glassman, Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models, in: Chi conference on human factors in computing systems extended abstracts, 2022, pp. 1–7.

[16] DeepSeek, Deepseek coder: Let the code write itself, https://github.com/deepseek-ai/DeepSeek-Coder (2023).

[17] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al., Code llama: Open foundation models for code, arXiv preprint arXiv:2308.12950.

[18] Y. Wei, Z. Wang, J. Liu, Y. Ding, L. Zhang, Magicoder: Source code is all you need, arXiv preprint arXiv:2312.02120.

[19] F. F. Xu, U. Alon, G. Neubig, V. J. Hellendoorn, A systematic evaluation of large language models of code, in: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, 2022, pp. 1–10.

[20] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, et al., A survey on evaluation of large language models, ACM Transactions on Intelligent Systems and Technology 15 (3) (2024) 1–45.

[21] J. Chen, Z. Pan, X. Hu, Z. Li, G. Li, X. Xia, Evaluating large language models with runtime behavior of program execution, arXiv preprint arXiv:2403.16437.

[22] T.-T. Nguyen, T. T. Vu, H. D. Vo, S. Nguyen, An empirical study on capability of large language models in understanding code semantics, arXiv preprint arXiv:2407.03611.

[23] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, arXiv preprint arXiv:2303.18223.

[24] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Advances in neural information processing systems 33 (2020) 1877–1901.

[25] S. Zhao, T. Nguyen, A. Grover, Probing the decision boundaries of in-context learning in llms, in: ICML 2024 Workshop on In-Context Learning.

[26] M. Jin, Q. Yu, J. Huang, Q. Zeng, Z. Wang, W. Hua, H. Zhao, K. Mei, Y. Meng, K. Ding, et al., Exploring concept depth: How large language models acquire knowledge at different layers?, arXiv preprint arXiv:2404.07066.

[27] Z. Song, S. Huang, Y. Wu, Z. Kang, Layer importance and hallucination analysis in large language models via enhanced activation variance-sparsity, arXiv preprint arXiv:2411.10069.

[28] F. Yin, J. Srinivasa, K.-W. Chang, Characterizing truthfulness in large language model generations with local intrinsic dimension, in: Forty-first International Conference on Machine Learning, 2024.

[29] M. Yuksekgonul, V. Chandrasekaran, E. Jones, S. Gunasekar, R. Naik, H. Palangi, E. Kamar, B. Nushi, Attention satisfies: A constraint-satisfaction lens on factual errors of language models, in: The Twelfth International Conference on Learning Representations, 2023.

[30] B. Snyder, M. Moisescu, M. B. Zafar, On early detection of hallucinations in factual question answering, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 2721–2732.

[31] D. Gottesman, M. Geva, Estimating knowledge in large language models without generating a single token, arXiv preprint arXiv:2406.12673.

[32] A. Slobodkin, O. Goldman, A. Caciularu, I. Dagan, S. Ravfogel, The curious case of hallucinatory (un) answerability: Finding truths in the hidden states of over-confident large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 3607–3625.

[33] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374.

[34] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al., Program synthesis with large language models, arXiv preprint arXiv:2108.07732.

[35] J. Li, G. Li, Y. Zhao, Y. Li, H. Liu, H. Zhu, L. Wang, K. Liu, Z. Fang, L. Wang, J. Ding, X. Zhang, Y. Zhu, Y. Dong, Z. Jin, B. Li, F. Huang, Y. Li, B. Gu, M. Yang, DevEval: A manually-annotated code generation benchmark aligned with real-world code repositories, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 3603–3614. doi:10.18653/v1/2024.findings-acl.214.
URL https://aclanthology.org/2024.findings-acl.214/

[36] T.-D. Bui, T. T. Vu, T.-T. Nguyen, S. Nguyen, D. H. Vo, Correctness assessment of code generated by large language models using internal representations.
URL https://github.com/iSE-UET-VNU/OPENIA

[37] Z. Ji, D. Chen, E. Ishii, S. Cahyawijaya, Y. Bang, B. Wilie, P. Fung, Llm internal states reveal hallucination risk faced with a query, in: Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP, 2024, pp. 88–104.

[38] Y.-S. Chuang, L. Qiu, C.-Y. Hsieh, R. Krishna, Y. Kim, J. Glass, Lookback lens: Detecting and mitigating contextual hallucinations in large language models using only attention maps, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 1419–1436.

[39] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, J. Ye, Inside: Llms' internal states retain the power of hallucination detection, in: The Twelfth International Conference on Learning Representations.

[40] G. Sriramanan, S. Bharti, V. S. Sadasivan, S. Saha, P. Kattakinda, S. Feizi, Llm-check: Investigating detection of hallucinations in large language models, Advances in Neural Information Processing Systems 37 (2024) 34188–34216.

[41] Z. Ding, H. Li, W. Shang, T.-H. P. Chen, Can pre-trained code embeddings improve model performance? revisiting the use of code embeddings in software engineering tasks, Empirical Software Engineering 27 (3) (2022) 1–38.

[42] Y. Zhao, L. Gong, Z. Huang, Y. Wang, M. Wei, F. Wu, Coding-ptms: How to find optimal code pre-trained models for code embedding in vulnerability detection?, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1732–1744.

[43] C. Niu, C. Li, B. Luo, V. Ng, Deep learning meets software engineering: A survey on pre-trained models of source code, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22, 2022, pp. 5546–5555.

[44] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 1536–1547.

[45] Y. Wang, W. Wang, S. Joty, S. C. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021, pp. 8696–8708.

[46] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, S. Hoi, Codet5+: Open code large language models for code understanding and generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 1069–1088.

[47] T.-D. Bui, D.-T. Luu-Van, T.-P. Nguyen, T.-T. Nguyen, S. Nguyen, H. D. Vo, Rambo: Enhancing rag-based repository-level method body completion, arXiv preprint arXiv:2409.15204.

[48] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, W. Chen, Repocoder: Repository-level code completion through iterative retrieval and generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 2471–2484.

[49] A. Hindle, E. T. Barr, M. Gabel, Z. Su, P. Devanbu, On the naturalness of software, Communications of the ACM 59 (5) (2016) 122–131.

[50] R.-M. Karampatsis, H. Babii, R. Robbes, C. Sutton, A. Janes, Big code!= big vocabulary: Open-vocabulary models for source code, in: 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE), IEEE, 2020, pp. 1073–1085.

[51] S. Nguyen, T. Nguyen, Y. Li, S. Wang, Combining program analysis and statistical language model for code statement completion, in: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2019, pp. 710–721.

[52] S. Nguyen, C. T. Manh, K. T. Tran, T. M. Nguyen, T.-T. Nguyen, K.-T. Ngo, H. D. Vo, Arist: An effective api argument recommendation approach, Journal of Systems and Software (2023) 111786.

[53] Z. Tu, Z. Su, P. Devanbu, On the localness of software, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2014, pp. 269–280.

[54] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, M. Lewis, Incoder: A generative model for code infilling and synthesis, arXiv preprint arXiv:2204.05999.

[55] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J. M. Zhang, Large language models for software engineering: Survey and open problems, in: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), IEEE, 2023, pp. 31–53.

[56] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, W. Chen, Repocoder: Repository-level code completion through iterative retrieval and generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 2471–2484.

[57] N. Jiang, K. Liu, T. Lutellier, L. Tan, Impact of code language models on automated program repair, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 1430–1442.

[58] M. Schäfer, S. Nadi, A. Eghbali, F. Tip, An empirical evaluation of using large language models for automated unit test generation, IEEE Transactions on Software Engineering.

[59] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Li, T. Su, Z. Yang, J. Tang, Codegeex: A pre-trained model for code generation with multilingual benchmarking

on humaneval-x, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023, pp. 5673–5684.

[60] S. Lu, N. Duan, H. Han, D. Guo, S.-w. Hwang, A. Svyatkovskiy, Reacc: A retrieval-augmented code completion framework, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2022, pp. 6227–6240.

[61] Z. Tang, J. Ge, S. Liu, T. Zhu, T. Xu, L. Huang, B. Luo, Domain adaptive code completion via language models and decoupled domain databases, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2023, pp. 421–433.

[62] F. Tambon, A. M. Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, G. Antoniol, Bugs in large language models generated code, arXiv preprint arXiv:2403.08937.

[63] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, X. Ma, Repoformer: Selective retrieval for repository-level code completion, in: Forty-first International Conference on Machine Learning.

[64] H. N. Phan, H. N. Phan, T. N. Nguyen, N. D. Bui, Repohyper: Better context retrieval is all you need for repository-level code completion, arXiv preprint arXiv:2403.06095.

[65] Y. Wang, Y. Wang, D. Guo, J. Chen, R. Zhang, Y. Ma, Z. Zheng, Rlcoder: Reinforcement learning for repository-level code completion, arXiv preprint arXiv:2407.19487.

[66] D. Song, Z. Zhou, Z. Wang, Y. Huang, S. Chen, B. Kou, L. Ma, T. Zhang, An empirical study of code generation errors made by large language models.

[67] A. Mohsin, H. Janicke, A. Wood, I. H. Sarker, L. Maglaras, N. Janjua, Can we trust large language models generated code? a framework for in-context learning, security patterns, and code evaluations across diverse llms, arXiv preprint arXiv:2406.12513.

[68] C. Spiess, D. Gros, K. S. Pai, M. Pradel, M. R. I. Rabin, A. Alipour, S. Jha, P. Devanbu, T. Ahmed, Calibration and correctness of language models for code, arXiv preprint arXiv:2402.02047.

[69] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, J. Chen, Security weaknesses of copilot generated code in github, arXiv preprint arXiv:2310.02059.

[70] Z. Zhang, Y. Wang, C. Wang, J. Chen, Z. Zheng, Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation, arXiv preprint arXiv:2409.20550.

[71] J. Wang, X. Luo, L. Cao, H. He, H. Huang, J. Xie, A. Jatowt, Y. Cai, Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval, arXiv preprint arXiv:2407.02395.

[72] S. Hamer, M. d'Amorim, L. Williams, Just another copy and paste? comparing the security vulnerabilities of chatgpt generated code and stackoverflow answers, in: 2024 IEEE Security and Privacy Workshops (SPW), IEEE, 2024, pp. 87–94.

[73] B. Kou, S. Chen, Z. Wang, L. Ma, T. Zhang, Do large language models pay similar attention like human programmers when generating code?, Proceedings of the ACM on Software Engineering 1 (FSE) (2024) 2261–2284.

[74] Y. Huang, J. Song, Z. Wang, S. Zhao, H. Chen, F. Juefei-Xu, L. Ma, Look before you leap: An exploratory study of uncertainty measurement for large language models, arXiv preprint arXiv:2307.10236.

[75] A. Nunez, N. T. Islam, S. K. Jha, P. Najafirad, Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing, arXiv preprint arXiv:2409.10737.

[76] A. Kavian, M. M. Pourhashem Kallehbasti, S. Kazemi, E. Firouzi, M. Ghafari, Llm security guard for code, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, 2024, pp. 600–603.

[77] G. Lin, S. Wen, Q.-L. Han, J. Zhang, Y. Xiang, Software vulnerability detection using deep neural networks: a survey, Proceedings of the IEEE 108 (10) (2020) 1825–1848.

[78] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, Poster: Vulnerability discovery with function representation learning from unlabeled projects, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2539–2541.

[79] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, Vuldeepecker: A deep learning-based system for vulnerability detection, in: 25th Annual Network and Distributed System Security Symposium, The Internet Society, 2018.

[80] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, Y. Wu, Vulsniper: focus your attention to shoot fine-grained vulnerabilities, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, 2019, pp. 4665–4671.

[81] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Advances in neural information processing systems 32.

[82] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet, IEEE Transactions on Software Engineering.

[83] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, C. Tao, Mvd: Memory-related vulnerability detection based on flow-sensitive graph neural networks, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 1456–1468.

[84] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 519–531.

[85] H. D. Vo, S. Nguyen, Can an old fashioned feature extraction and a light-weight model improve vulnerability type identification performance?, Information and Software Technology (2023) 107304.

[86] S. Nguyen, T.-T. Nguyen, T. T. Vu, T.-D. Do, K.-T. Ngo, H. D. Vo, Code-centric learning-based just-in-time vulnerability detection, Journal of Systems and Software 214 (2024) 112014.

[87] H. D. Vo, S. Nguyen, Can an old fashioned feature extraction and a light-weight model improve vulnerability type identification performance?, Information and Software Technology 164 (2023) 107304.

[88] Y. Li, S. Wang, T. N. Nguyen, S. Van Nguyen, Improving bug detection via context-based code representation learning and attention-based neural networks, Proceedings of the ACM on Programming Languages 3 (OOPSLA) (2019) 1–30.

[89] T.-T. Nguyen, H. D. Vo, Context-based statement-level vulnerability localization, Information and Software Technology 169 (2024) 107406.

[90] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, Sysevr: A framework for using deep learning to detect software vulnerabilities, IEEE Transactions on Dependable and Secure Computing.

[91] Y. Li, S. Wang, T. N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 292–303.

[92] Y. Ding, S. Suneja, Y. Zheng, J. Laredo, A. Morari, G. Kaiser, B. Ray, Velvet: a novel ensemble learning approach to automatically locate vulnerable statements, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering, IEEE, 2022, pp. 959–970.

[93] M. Fu, C. Tantithamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 608–620.

[94] D. Hin, A. Kan, H. Chen, M. A. Babar, Linevd: Statement-level vulnerability detection using graph neural networks, in: IEEE/ACM 19th International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022, IEEE, 2022, pp. 596–607.

[95] Y. Huang, L. Sun, H. Wang, S. Wu, Q. Zhang, Y. Li, C. Gao, Y. Huang, W. Lyu, Y. Zhang, et al., Position: Trustllm: Trustworthiness in large language models, in: International Conference on Machine Learning, PMLR, 2024, pp. 20166–20270.

[96] J. Ferrando, G. Sarti, A. Bisazza, M. R. Costa-jussà, A primer on the inner workings of transformer-based language models, arXiv preprint arXiv:2405.00208.
[97] A. Azaria, T. Mitchell, The internal state of an llm knows when it's lying, in: The 2023 Conference on Empirical Methods in Natural Language Processing.
[98] J. He, Y. Gong, Z. Lin, Y. Zhao, K. Chen, et al., Llm factoscope: Uncovering llms' factual discernment through measuring inner states, in: Findings of the Association for Computational Linguistics ACL 2024, 2024, pp. 10218–10230.
[99] H. Kim, A. Bibi, P. Torr, Y. Gal, Detecting llm hallucination through layer-wise information deficiency: Analysis of unanswerable questions and ambiguous prompts, arXiv preprint arXiv:2412.10246.
[100] Z. Chu, Y. Wang, L. Li, Z. Wang, Z. Qin, K. Ren, A causal explainable guardrails for large language models, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 1136–1150.
[101] E. Fadeeva, A. Rubashevskii, A. Shelmanov, S. Petrakov, H. Li, H. Mubarak, E. Tsymbalov, G. Kuzmin, A. Panchenko, T. Baldwin, et al., Fact-checking the output of large language models via token-level uncertainty quantification, arXiv preprint arXiv:2403.04696.
[102] K. Li, O. Patel, F. Viégas, H. Pfister, M. Wattenberg, Inference-time intervention: Eliciting truthful answers from a language model, Advances in Neural Information Processing Systems 36.
[103] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, Advances in neural information processing systems 35 (2022) 24824–24837.